FixedIncomeLib

Neel Shah
New York University

February 16, 2026

Overview

FixedIncomeLib is a Python framework that spans every stage of fixed-income analytics, from raw market data to valuations and first-order risk. In this monograph, we document how the shipped code is structured and how its components interact:

- **Data Ingestion:** Creation of `Data1D` and `Data2D` objects and their registration into a `DataCollection`. `Data1D` is an `(axis, value)` series (factory input columns `AXIS1`, `VALUES`); `Data2D` is a pivoted surface (DataFrame index=`axis1`, columns=`axis2`). For curve calibration, `data/yc_market_loader.py` provides `build_yc_data_collection(value_date, market_df)` to convert a market table with columns `DATA TYPE`, `DATA CONVENTION`, `AXIS`, `VALUE` into a `DataCollection` of instrument quote series.
- **Date and Schedule Utilities:** The `Date` class plus `addPeriod`, `moveToBusinessDay`, and `accrued` implement QuantLib day-count and business-day conventions, and `makeSchedule`/`business_day_schedule` generate accrual/fixing/payment schedules used by products and valuation engines.
- **Market and Data Conventions:** Wrappers in `market/basics.py` map human-readable keys to QuantLib enums for day-count, calendars, business-day rules, indices, and currencies. The `conventions` package adds a JSON-backed `DataConventionRegistry` that maps instrument conventions (e.g. `USD-SOFR-OIS`, `SOFR-FUTURE-3M`) to the assumptions required to build products consistently.
- **Historical Fixings:** The singleton `IndexManager` in `valuation/index_fixing_registry.py` loads and serves past rate fixings from CSV, enabling realized-rate accrual in overnight instruments.
- **Numerical Methods:** `Interpolator1D` and `Interpolator2D` (in `utilities/numerics.py`) provide 1D and bilinear 2D interpolation. `utilities/optimization.py` provides a light-weight `simple_solver` used by curve calibration.
- **Model Interface and Risk Plumbing:** The abstract `Model` / `ModelComponent` types (in `model/model.py`) standardize how components calibrate, store state variables, expose gradients, and assemble a full model Jacobian. This is used by `utilities/risk_reporting.py` to map product risk from internal state variables back to the calibration instrument space.
- **Instrument Definitions:** Product classes for bullet cashflows, IBOR and overnight cashflows, fixed-float swaps, futures, caplets/floorlets, swaptions, and composite portfolios live in `product/*.py`, and are valued via the Visitor pattern.

- **Valuation Engines:** Pricing and first-order risk logic is implemented in `yield_curve/valuation_engine_yc.py` and `sabr/valuation_engine_sabr.py`. `valuation/valuation_engine_registry.py` dispatches `(model, product)` pairs to the correct engine.
- **Yield Curve Calibration:** `yield_curve/yield_curve_model.py` builds a curve by turning market quotes into calibration products (via the `builders` package), assembling a `CalibrationBasket`, generating anchor pillars, and solving piecewise-constant instantaneous forward-rate (IFR) buckets sequentially.
- **SABR Surfaces and Approaches:** `sabr/sabr_model.py` ingests and interpolates grids for `NORMALVOL`, `BETA`, `NU`, and `RHO`, with optional shift and time-decay metadata. The `analytics` package supports both "bottom-up" and "top-down" aggregation routines for RFR products.
- **Testing:** Jupyter notebooks in `tests/` exercise the library end-to-end: market ingestion, curve calibration, product valuation, and first-order risk checks.

Code fragments in this report are either taken directly from the library source or presented as explicit usage examples; where a snippet is a usage example, it is labeled as such.

Contents

Chapter 1

Introduction

In modern quantitative finance, fixed-income instruments ranging from simple coupon-bearing bonds to complex interest-rate derivatives form the bedrock of global financial markets. Their valuation and risk management require concerted workflows that integrate heterogeneous market data, robust mathematical models, and flexible software architectures. FixedIncomeLib was conceived to address these challenges by providing:

1. **Transparency**: All algorithms from day-count calculations and business-day adjustments to curve bootstrapping and SABR interpolation are implemented in clear, self-contained Python modules with full docstrings and test coverage.

2. **Modularity**: The code is organized into a set of core packages (`data`, `date`, `market`, `conventions`, `builders`, `utilities`, `model`, `product`, `analytics`, `yield_curve`, and `sabr`) plus the top-level `valuation` package. Each package encapsulates a single layer of functionality, enabling independent development and seamless substitution.

3. **Extensibility**: New market conventions, interpolation schemes, financial instruments, and pricing engines can be added without modifying existing modules. Extension points such as the `ValuationEngineRegistry` and base classes `Product` and `Model` facilitate plug-and-play integration.

This introduction establishes the core principles and design philosophy that underpin FixedIncomeLib. Subsequent chapters will build on these foundations, revealing the detailed implementation and usage patterns of each component.

To ground this monograph in reality, each chapter references exact source lines and modules from the delivered 1.0.0 release, providing both theoretical context and practical implementation insights.

Chapter 2

Architectural Design

2.1 High-Level Layer Diagram

Figure 2.1 shows how the core packages flow from raw data down to models, products, and valuation engines.

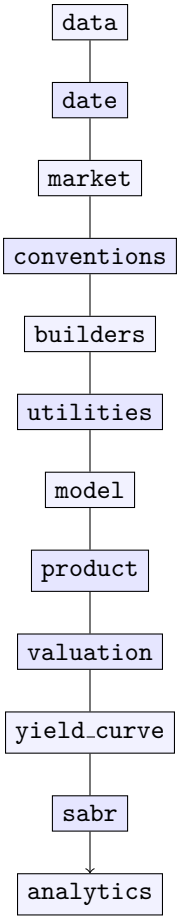

Figure 2.1: FixedIncomeLib's Package Dependency Flow (fixings are handled via `IndexManager` and CSV resources)

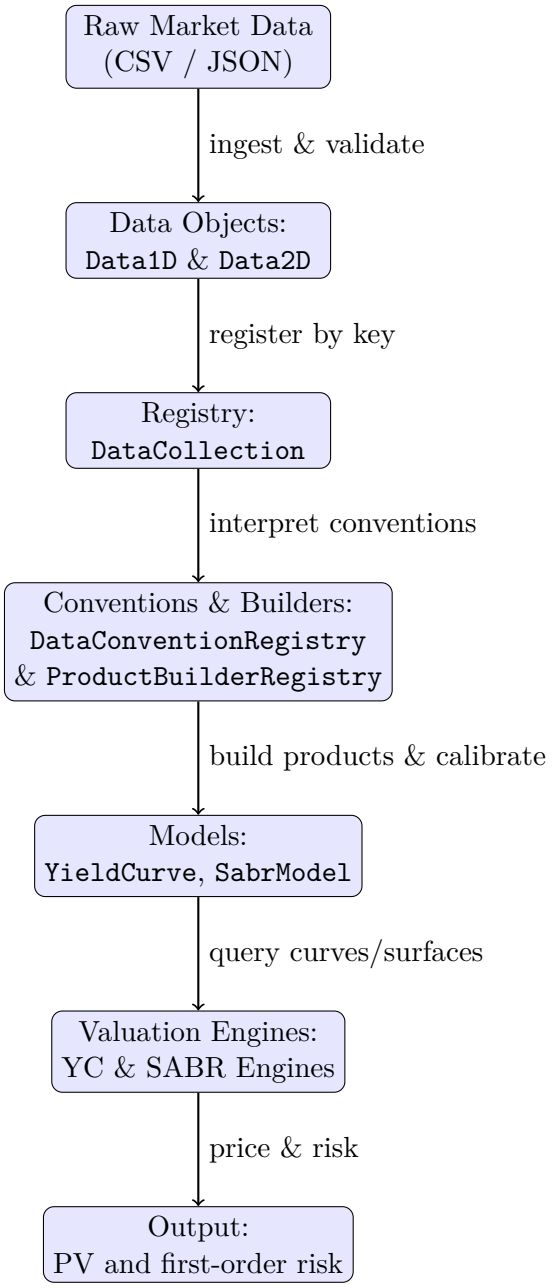

Figure 2.2: End-to-End Data-Flow in FixedIncomeLib

2.2 Quickstart (usage example)

The typical workflow is:

1. Load a "market table" into a DataFrame (curve quotes).
2. Convert it into a `DataCollection` with `build_yc_data_collection`.
3. Create and calibrate a `YieldCurve` with build methods.
4. For volatility, create `Data2D` surfaces for `NORMALVOL`, `BETA`, `NU`, `RHO`, register them in a `DataCollection`, then create a `SabrModel`.
5. Price and compute first-order risk using `ValuationEngineRegistry` (or `utilities/risk_reporting.py` convenience functions).

```python
import numpy as np
from fixedincomelib.data.yc_market_loader import build_yc_data_collection
from fixedincomelib.yield_curve import YieldCurve
from fixedincomelib.product.linear_products import ProductOvernightSwap
from fixedincomelib.valuation import ValuationEngineRegistry

# 1) Curve market data -> DataCollection
# market_df columns: ["DATA TYPE","DATA CONVENTION","AXIS","VALUE"]
dc_curve = build_yc_data_collection(value_date="2026-02-12", market_df=market_df)

# 2) Curve build method(s)
build_methods_yc = [{
    "TARGET": "SOFR-1B",
    "INSTRUMENTS": ["SOFR-FUTURE-3M", "USD-SOFR-OIS"],
    "INTERPOLATION METHOD": "PIECEWISE_CONSTANT",
    "LOCAL_TOL": 1e-10,}]

yc = YieldCurve("2026-02-12", dc_curve, build_methods_yc)

# 3) Price a product (example OIS)
ois = ProductOvernightSwap(
    effectiveDate="2026-02-13",
    maturityDate="2028-02-13",
    frequency="1Y",
    overnightIndex="SOFR-1B",
    fixedRate=0.03,
    notional=10_000_000,
    position="RECEIVE")

params = {"FUNDING INDEX": "SOFR-1B"}
ve = ValuationEngineRegistry().new_valuation_engine(yc, params, ois)

ve.calculateValue()
status, pv = ve.value_

ve.calculateFirstOrderRisk()
g_theta = np.asarray(ve.firstOrderRisk_, dtype=float)  # dPV/d(theta)
```

Listing 2.1: Minimal end-to-end example: curve calibration + PV + first-order risk

Chapter 3

Data Layer

The Data layer in FixedIncomeLib provides robust, type-safe abstractions for all market inputs. Its primary goals are:

- **Separation of Concerns**: Data ingestion and validation are isolated from modeling and valuation logic.
- **Type Safety**: Every dataset carries explicit metadata (`data_type`, `data_convention`) and is validated at construction.
- **Uniform Access**: A central registry (`DataCollection`) enforces unique keys and offers fast lookups by `(data_type, data_convention)`.

3.1 Core Interface: `MarketData`

All data objects inherit from the abstract base class `MarketData`, which defines:

- `data_type`: a short string that categorizes the data (examples in this repository include `"RFR FUTURE"`, `"RFR SWAP"` for curve instruments, and `"normalvol"`, `"beta"`, `"nu"`, `"rho"` for SABR surfaces).
- `data_convention`: a convention key (e.g. `"USD-LIBOR-BBA-3M"`).
- `key()`: returns the tuple `(data_type, data_convention)`, used for registry indexing.
- `__repr__()`: subclasses must provide a human-readable summary for debugging.

3.2 `Data1D`: One-Dimensional Series

`Data1D` encapsulates sequences of `(axis[i], values[i])`, typical use cases include:

- Deposit tenors versus deposit rates.
- Swap maturities versus swap rates.
- Historical fixing dates versus observed rates.

Key features:

- *Validation*: Constructor checks that the axis and value lists have equal length, raising `ValueError` otherwise.
- *Factory Method*: `Data1D.createDataObject(data_type, data_convention, df)` builds from a pandas DataFrame with columns `AXIS1` and `VALUES`, enforcing type and column presence.

3.3 `Data2D`: Two-Dimensional Surfaces

`Data2D` stores $N \times M$ matrices, for example:

- SABR volatility grids indexed by *expiry* and *tenor*.
- Correlation surfaces parameterized by two axes.

Key features:

- *Dimension Checks*: Validates that the number of rows matches `axis1` length and each row's length matches `axis2`.
- *Factory Method*: `Data2D.createDataObject(data_type, data_convention, df)` accepts a pivoted DataFrame (index=`axis1`, columns=`axis2`), converting its values to a NumPy array.
- *Representation*: `__repr__()` displays the data type, convention, and matrix shape.

3.4 `DataCollection`: Central Registry

`DataCollection` maintains an in-memory map from `(data_type, data_convention)` to `MarketData` instances:

- *Uniqueness*: The constructor enforces no duplicate keys, raising `KeyError` on collisions.
- *Lookup*: The method `get(data_type, data_convention)` retrieves the data object or fails fast if missing.
- *Reset*: `clear()` empties the registry, useful for isolated testing.

simply call:

```
dc.get("RFR SWAP","USD-SOFR-OIS")
```

to fetch a yield-curve series, without knowing how it was loaded.

Field	Details
Purpose	In-memory registry mapping `(dataType, dataConvention)` to `MarketData` objects. Ensures uniqueness and makes downstream code independent of the data-loading mechanism.
Key public methods	`DataCollection(list\_of\_MarketData)`; `addData(md)`; `getDataFromDataCollection(dataType, dataConvention)`; `keyExists(key)`; `getAllData()`
Inputs / outputs (units)	Inputs: a list of `MarketData` instances; output: a validated registry. Lookups return the stored object.

3.5 Typical Workflow

Two common ingestion patterns appear throughout the notebooks in `tests/`:

(A) Direct creation via `createDataObject`

1. **Load** raw market data into a pandas DataFrame.
2. **Instantiate** a data container:
   - `d1 = Data1D.createDataObject("RFR SWAP","USD-SOFR-OIS",df_curve)` for 1D term structures where `df_curve` has columns `AXIS1`, `VALUES`.
   - `d2 = Data2D.createDataObject("normalvol","SOFR-1B",df_surface)` for 2D surfaces where `df_surface` is already pivoted with index=`axis1` and columns=`axis2`.
3. **Register** the objects:

   `dc = DataCollection([d1, d2, ...])`.

4. **Consume** downstream:

   `nv = dc.get("normalvol","SOFR-1B")`.

(B) Yield-curve calibration ingestion via `build_yc_data_collection`

For yield-curve calibration, the notebooks typically start from a single "market table" with columns:

`[DATA TYPE, DATA CONVENTION, AXIS, VALUE]`.

Here, `DATA CONVENTION` is a convention key resolved by `DataConventionRegistry` (e.g. `SOFR-FUTURE-3M`, `USD-SOFR-OIS`), while `AXIS` is the instrument-specific identifier (e.g. `"YYYY-MM-DD x YYYY-MM-DD"` for RFR futures or `"5Y"` for swaps). The helper:

`build_yc_data_collection(value_date, market_df)`

groups rows by `DATA CONVENTION` and produces a `DataCollection` whose entries are `Data1D` series of `(AXIS, VALUE)` pairs for each convention.

By centralizing data handling here, FixedIncomeLib ensures consistency, testability, and clear separation between data ingestion and financial logic.

Chapter 4

Date Layer

The Date layer centralizes all calendar, period and day-count operations, which are critical for accurate accrual, discounting, and schedule generation. By wrapping QuantLib conventions in concise Python classes and functions, this layer guarantees consistent treatment of dates across bootstrapping, cashflow generation, and option-expiry handling.

Key responsibilities:

- **Parsing and Conversion**: Accept ISO strings (`"YYYY-MM-DD"`), `datetime.date` or QuantLib's own `Date` and normalize them to our unified `Date` subclass.
- **Business-Day Adjustments**: Shift dates according to conventions like `FOLLOWING`, `MODIFIED_FOLLOWING`, using named calendars (e.g., TARGET, US).
- **Period Arithmetic**: Add tenors (e.g., "3M", "6M", "1Y") with correct roll and end-of-month rules.
- **Year-Fraction Calculations**: Compute accrual factors under conventions such as `ACT/360`, `ACT/365`, and `30/360`.

4.1 Design Rationale

Rather than scattering date logic throughout the codebase, FixedIncomeLib consolidates it here:

1. A minimal `Date` subclass (in `date/classes.py`) that hides QuantLib's verbose constructors and accepts multiple input types.
2. Stateless utility functions (in `date/utilities.py`) for all adjustments, accruals, and schedule generation.
3. A single, internal mapping of supported conventions and calendars, ensuring any update applies globally.

This approach produces cleaner model and valuation code, and guarantees that a change in, say, the ACT/360 rule propagates everywhere instantly.

4.2 Core Classes (`date/classes.py`)

4.2.1 `Date` (subclass of `ql.Date`)

- Accepts one argument of type:
  - ISO string (`"2025-08-01"`)
  - `datetime.date`
  - existing `ql.Date`
- Normalizes all to QuantLib's `Date(day,month,year)` internally.
- Eliminates parsing boilerplate elsewhere.

4.2.2 `Period` (alias of `ql.Period`)

- Renamed for symmetry with `Date`.
- Used for tenor arithmetic without importing QuantLib directly.

4.2.3 `TermOrTerminationDate`

- Given a string, decides whether it denotes a term (e.g. "3M") or a termination date ("YYYY-MM-DD").
- Exposes:
  - `isTerm()` – true if it holds a tenor.
  - `getTerm()` – returns the `Period`.
  - `getDate()` – returns the `Date`.
- Simplifies APIs that accept either a tenor or a fixed date.

4.3 Utility Functions (`date/utilities.py`)

A small set of pure functions drives all calendar logic:

- `addPeriod(start_date, term, biz_conv, hol_conv, endOfMonth)` Advances `start_date` by `term` (e.g. "6M"), applies business-day and holiday conventions, and handles end-of-month rules.
- `moveToBusinessDay(date, biz_conv, hol_conv)` Adjusts any date to a valid business day per specified conventions.
- `accrued(start, end, accrual_basis, biz_conv, hol_conv)` First moves the end-date to a business day, then computes the year-fraction with conventions like ACT/360, ACT/365, 30/360.
- `makeSchedule(...)` Generates a full cashflow schedule DataFrame (start/end dates, fixing/-payment dates, accrual factors) for any leg frequency.

Each function relies on our `Date` and `Period` classes plus a single, internal mapping of QuantLib day-count and holiday conventions.

4.4 Usage Example

Here is a typical snippet from a pricing notebook, illustrating accrual and schedule creation:

```
from fixedincomelib.date.classes import Date, Period, TermOrTerminationDate
from fixedincomelib.date.utilities import addPeriod, moveToBusinessDay, accrued,
    makeSchedule

# 1) Normalize input strings to our Date subclass
start = Date("2025-01-15")
end   = Date("2026-01-15")

# 2) Advance by 6 months, modified-following on TARGET calendar
six_mo_later = addPeriod(start, "6M", biz_conv="MF", hol_conv="TARGET")

# 3) Compute ACT/360 accrual factor
alpha = accrued(start.isoformat(), six_mo_later.isoformat(),
                accrual_basis="ACT/360", biz_conv="MF", hol_conv="TARGET")
# alpha == 0.5

# 4) Build semi-annual cashflow schedule DataFrame
schedule_df = makeSchedule(
    start_dt       = start.isoformat(),
    end_dt         = end.isoformat(),
    frequency      = "6M",
    hol_conv       = "TARGET",
    biz_conv       = "MF",
    acc_basis      = "ACT/360",
    rule           = "FORWARD",
    endOfMonth     = True,
    fix_in_arrear  = False,
    fixing_offset  = "2B",
    payment_offset = "2B"
)

print(schedule_df.head())
```

Listing 4.1: Date-layer operations in practice

By consolidating all date parsing, adjustment, and accrual logic in this chapter, FixedIncomeLib ensures that every model and valuation engine downstream uses identical, thoroughly tested rules.

Chapter 5

Market Layer

The Market layer standardizes all market conventions like day-count, compounding, calendars, currencies, and rate indices. By isolating these concerns, FixedIncomeLib ensures that every model and valuation engine uses identical, data-driven parameters without duplicating logic or hard-coding strings.

5.1 Design Rationale

- **Thin Wrappers**: Each convention is a lightweight Python class around a QuantLib enum or object, exposing a clean, string-based API.
- **Central Registry**: Rate indices (e.g. USD-LIBOR-BBA, SONIA-1B) are loaded from `market/index_registry.json` into an `IndexRegistry` singleton, avoiding hard-coded mappings.
- **Fail-Fast**: Invalid convention names or missing indices trigger clear exceptions immediately, aiding data QA.

5.2 Convention Wrappers (`market/basics.py`)

Rather than scattering QuantLib imports and enums, three classes encapsulate the most common conventions:

`BusinessDayConvention(input):` Maps inputs `"MF"`, `"F"`, `"P"` to QuantLib's `ModifiedFollowing`, `Following`, and `Preceding` enums.

`HolidayConvention(input):` Provides named calendars (e.g. `"TARGET"`, `"NYC"`, `"LON"`) via QuantLib's `JointCalendar`, `UnitedStates`, `UnitedKingdom`, etc.

`AccrualBasis(input):` Wraps QuantLib day-count conventions (`ACT/360`, `ACT/365 FIXED`, `30/360`, `BUSINESS252`, `ACT/ACT`).

Usage example:

```
from fixedincomelib.market.basics import BusinessDayConvention, HolidayConvention,
    AccrualBasis

# Modified-Following on TARGET calendar:
biz_conv = BusinessDayConvention("MF").value
hol_conv = HolidayConvention("TARGET").value

# ACT/365 Fixed day count:
day_counter = AccrualBasis("ACT/365 FIXED").value
```

Each wrapper validates its input string and raises an exception for unsupported values.

5.3 Currency and Index Registry (`market/indices.py` & JSON)

5.3.1 `Currency(input)`

Encapsulates common ISO currency codes:

- `"USD"` → `ql.USDCurrency()`
- `"EUR"` → `ql.EURCurrency()`

Invalid codes produce a clear `Exception`.

5.3.2 `IndexRegistry`

- On first instantiation, reads `market/index_registry.json`, which maps keys like `"USD-LIBOR-BBA"` to QuantLib class names (`"USDLibor"`, `"Sonia"`).
- Builds `RegistryMap:{ key → QuantLibClass }`.
- `get(key, tenor?)` returns a daily index or a tenor-specific index (e.g. `ql.USDLibor(ql.Period(3,ql.Months))`).

Usage example:

```
from fixedincomelib.market.indices import indexReg

# Daily USD-LIBOR index
libor = indexReg.get("USD-LIBOR-BBA")

# 6-month USD-LIBOR index
libor6m = indexReg.get("USD-LIBOR-BBA", 6)
```

A missing key raises `KeyError`, preventing silent failures.

5.4 Data Conventions (`conventions/`)

While the Market layer maps general calendar/day-count/index primitives, curve and volatility construction also needs *instrument conventions*: a compact description of how to interpret each quoted instrument type. FixedIncomeLib stores these in `conventions/data_convention_registry.json` and exposes them via:

5.4.1 DataConventionRegistry

- Lazily loads the JSON registry on first use.
- `get(name)` returns a `DataConvention` instance for the given convention key.
- A missing convention raises `KeyError`, preventing accidental silent fallbacks.

5.4.2 DataConvention

A `DataConvention` captures the assumptions required to build a product consistently from a market quote. Examples present in the shipped registry include:

- `SOFR-FUTURE-3M`: used to build RFR futures from "start x end" date ranges.
- `USD-SOFR-OIS`: used to build an overnight indexed swap (OIS) from a maturity term.

Each convention contains a `kind` (used by the builder layer to select the appropriate product builder) plus fields like `index`, `discounting`, `currency`, `accrualBasis`, and business-day/holiday conventions.

These conventions are consumed by the `builders` package when transforming a `DataCollection` of calibration quotes into concrete `Product` instances.

5.4.3 Builders

Builders are the bridge between *quotes* and *products*. Given a quote series and a resolved `DataConvention`, builders construct the correct `Product` object with the correct schedule and conventions.

5.5 ProductBuilderRegistry (fixedincomelib/builders/product_builder_regi

Field	Details
Purpose	Dispatch from a `DataConvention` subclass to a product builder function.
Key public methods	`ProductBuilderRegistry().insert(conventionType, builderFn);` `ProductBuilderRegistry().get_builder(convention)`
Inputs / outputs (units)	Input: a convention instance. Output: a callable builder that returns a `Product`.

5.6 Building a curve calibration basket (fixedincomelib/builders/basket_bui

`build_yc_calibration_basket_from_dc(valueDate, dataCollection, buildMethod)` constructs a `CalibrationBasket` by:

1. selecting `Data1D` series from `DataCollection` (based on `buildMethod["INSTRUMENTS"]`),
2. resolving each convention key through `DataConventionRegistry`,
3. for each row/axis value, invoking the correct product builder, and
4. storing `(product, quote, metadata)` as `CalibItem` elements.

5.7 Anchor pillars (`fixedincomelib/builders/pillar_builders.py`)

`build_anchor_pillars(calibrationBasket, max_pillar_date=None)` derives a strictly increasing pillar date set from calibration instruments (typically using their termination/maturity dates) and wraps them into `PillarNode` objects that hold the node id, date, time, associated instrument, and the IFR state variable solved for that node.

5.8 Summary

By centralizing market conventions in `market/` and fixings in `valuation/index_fixing_registry.py`, FixedIncomeLib:

- Ensures all modules-models, products, and valuation engines-share identical conventions and historical data.
- Provides easy extension: to add a new convention or index, update one JSON or one wrapper class.
- Delivers fail-fast behavior on invalid inputs, improving data integrity in production and research.
- Product Builders and Basket Builders are used in the creation of products for calibration of yield curve.

Chapter 6

Utilities Layer

The Utilities layer provides the core numerical building blocks used throughout FixedIncomeLib. This is specifically, fast, transparent interpolation and integration routines. By isolating these algorithms in a single module, we ensure:

- **Reusability**: All packages needing interpolation (curves, surfaces, risk metrics) share the same, well–tested code.
- **Auditability**: The implementations are kept under 100 lines and avoid hidden dependencies.
- **Predictable Performance**: Complexity characteristics are documented and validated, allowing higher–level modules to budget accordingly.

6.1 Design Rationale

Instead of relying on black box numerical libraries, FixedIncomeLib implements:

1. *Piecewise constant interpolation* (1D) for fastest lookup in situations where step functions are sufficient.
2. *Bilinear interpolation* (2D) for smooth surfaces like SABR parameter grids.
3. *Definite integrals* over piecewise constant functions, enabling analytic accumulation of values (e.g. forward rate integrals).

This deliberate simplicity guarantees that every result can be traced back to clear, linear time or logarithmic time code.

6.2 Module `utilities/numerics.py`

6.2.1 Interpolator1D (Piecewise Constant)

- **Constructor**:
  - `axis`: strictly increasing list of breakpoints $\{x_i\}$.
  - `values`: list of corresponding levels $\{v_i\}$.
  - Enforces `method=="PIECEWISE_CONSTANT"` and $|\texttt{axis}| = |\texttt{values}|$.

- **interpolate(time)**:
  - If $t < x_0$, returns v_0.
  - If $x_{i-1} \le t < x_i$, returns v_i.
  - If $t \ge x_{n-1}$, returns v_{n-1}.
- **integral(start,end)**:
  - Analytically sums $\int_s^e v(x)\,dx$ across intervals in $O(n)$ time.

Complexity: `interpolate` is $O(n)$ in worst case (linear scan). `integral` is $O(n)$.

6.2.2 Interpolator2D (Bilinear)

- **Constructor**:
  - `axis1`, `axis2`: 1D numpy arrays of grid points.
  - `values`: 2D numpy array of shape $|\texttt{axis1}| \times |\texttt{axis2}|$.
  - Enforces `method=="LINEAR"`.
- **interpolate(x,y)**:
  1. Clamp (x, y) to the bounding rectangle.
  2. Locate indices i, j via binary search (`np.searchsorted`).
  3. Compute the bilinear formula on the surrounding four grid values.

Complexity: Index searches are $O(\log n + \log m)$; arithmetic is $O(1)$.

6.3 Usage Example

```
from fixedincomelib.utilities.numerics import Interpolator1D, Interpolator2D
import numpy as np

# 1D piecewise-constant
axis = [0.0, 1.0, 2.0, 5.0]
values = [0.02, 0.025, 0.03, 0.035]
i1 = Interpolator1D(axis, values, method="PIECEWISE_CONSTANT")
r1 = i1.interpolate(1.7)      # returns 0.03
area = i1.integral(0.5, 3.5)  # analytic sum

# 2D bilinear
a1 = np.array([0.5, 1.0, 2.0])
a2 = np.array([0.01, 0.02, 0.03, 0.04])
grid = np.random.rand(3,4)  # e.g., SABR alpha surface
i2 = Interpolator2D(a1, a2, grid, method="LINEAR")
vol = i2.interpolate(1.3, 0.025)
```

Listing 6.1: 1D and 2D interpolation usage

6.4 Root Solvers (`utilities/optimization.py`)

Curve calibration repeatedly solves one-dimensional root-finding problems of the form

$$\texttt{residual}(\theta_k) = 0,$$

where a single bucket parameter θ_k is adjusted until a calibration instrument matches its market quote. FixedIncomeLib provides a minimal secant-based solver:

6.4.1 `simple_solver`

`simple_solver(residual_fn, x_prev, x_curr, tolerance, max_iter)` implements the secant iteration

$$x_{n+1} = x_n - \ f(x_n)\,\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})},$$

with a relative stopping condition `|x_next-x_curr| <= tol*(1+|x_curr|)`. This function is used in `yield_curve/yield_curve_model.py` during per-pillar bootstrapping.

6.5 Risk Reporting Utilities (`utilities/risk_reporting.py`)

Beyond per-parameter gradients inside models, the library includes convenience helpers to produce desk-friendly reports in a single call.

6.5.1 `create_value_report`

Creates a valuation engine for a given `(model, product)` pair via `ValuationEngineRegistry`, runs `calculateValue()`, and returns PV (and optionally other engine outputs depending on the product).

6.5.2 `create_risk_report`

Runs `calculateFirstOrderRisk()` for the product to produce the internal model gradient $g = \partial PV/\partial\theta$. It then maps this internal risk into calibration-instrument space by solving

$$J^\top r = g,$$

where J is the model Jacobian returned by `model.jacobian()`. Finally, the result is scaled by `model.calibration_quote_sensitivity()` when the model provides a quote-to-PV scaling for each calibration pillar.

6.5.3 Usage Example

```
import numpy as np
from fixedincomelib.utilities.risk_reporting import (
    create_value_report, create_risk_report
)
from fixedincomelib.yield_curve import YieldCurve
from fixedincomelib.product import ProductOvernightSwap

# model: assume already calibrated
yc = YieldCurve(value_date, dc, build_methods)

# product
ois = ProductOvernightSwap(...)

pv_report   = create_value_report({"FUNDING INDEX": "SOFR-1B"}, yc, ois)
risk_report = create_risk_report({"FUNDING INDEX": "SOFR-1B"}, yc, ois)
```

Listing 6.2: PV and risk reports in one call

By centralizing these routines, FixedIncomeLib guarantees that all curve interpolations and surface lookups behave identically and perform predictably across the entire framework.

Chapter 7

Model Layer

The Model layer defines the abstractions that connect market data to valuation. A `Model` owns one or more `ModelComponent` objects, each component encapsulating a coherent parameter block (curve pillars, surface grids, *etc.*). Products and valuation engines query models for quantities like discount factors, forwards, or interpolated surface parameters; risk flows in the opposite direction via gradients and Jacobians.

In the shipped repository, the main concrete models are:

- `YieldCurve` (in `yield_curve/yield_curve_model.py`)
- `SabrModel` (in `sabr/sabr_model.py`)

7.1 Design Rationale

Models in FixedIncomeLib follow a common pattern:

1. **Components own state:** Each `ModelComponent` stores its calibrated state variables `stateVars_` (e.g. IFR buckets, SABR surface grids) and, when asked, produces gradients into a model-level `gradient_` array.
2. **Models:** A `Model` is responsible for constructing components from `buildMethodCollection`, exposing convenience query methods (e.g. `discountFactor`), and assembling a full Jacobian against its calibration instruments.
3. **Risk is standardized:** Valuation engines push derivatives into the model via component-gradient methods; reporting utilities can then map risk back to calibration-instrument space using the model Jacobian.

This separation keeps product valuation code focused on financial logic, while calibration, interpolation, and parameter management remain centralized and testable.

7.2 Core Interfaces (`model/model.py`)

7.2.1 `Model`

The abstract base class `Model` defines the minimal API expected by the valuation layer:

- `valueDate`: model valuation date.

- `components_`: ordered dict of model components.
- `newModelComponent(build_method)`: factory for a component based on a build method dict.
- `retrieveComponent(componentKey)`: return a component by key.
- `clearGradient()` / `getGradientArray()`: manage and flatten the model gradient vector.
- `jacobian()`: (implemented by concrete models) return the Jacobian of calibration-instrument PVs w.r.t. model state variables.
- `calibration_quote_sensitivity()`: (optional) return the mapping from quote units to PV units for each calibration pillar.

7.2.2 `ModelComponent`

A `ModelComponent` encapsulates a named parameter block. The base class provides:

- `target_`: a short identifier for the component (e.g. an index name).
- `buildMethod_`: the build method dict used to configure the component.
- `stateVars_`: calibrated state variables for this component.
- `calibrate()`: the component-specific calibration routine.
- `getStateVariables()` / `setStateVariables()`.

Concrete components (e.g. `YieldCurveModelComponent`, `SabrModelComponent`) implement calibration and any interpolation/weighting logic needed for fast model queries and gradients.

7.3 Build Methods

A build method is a plain Python dict. The model layer interprets it to decide:

1. what components to create,
2. how to calibrate them, and
3. how to interpolate/extrapolate between pillar points.

7.3.1 Yield curve build methods

For `YieldCurve`, each component corresponds to one target curve (e.g. `"SOFR-1B"`). The build method keys used by `YieldCurveModelComponent` include:

- `TARGET` (required): the curve/index name.
- `INSTRUMENTS` (required): a list of `DATA CONVENTION` keys to use when building the calibration basket (e.g. `["SOFR-FUTURE-3M","USD-SOFR-OIS"]`).
- `INTERPOLATION METHOD` (optional): currently `PIECEWISE_CONSTANT` is used for IFR buckets.
- `LOCAL_TOL` (optional): local tolerance for the per-pillar solver.

7.3.2 SABR build methods

For `SabrModel`, each component corresponds to an index plus a `VALUES` tag (and optionally a `PRODUCT` tag). The component ingests the same underlying SABR surfaces (`normalvol`, `beta`, `nu`, `rho`) but may attach different metadata depending on how the surfaces are used. Keys consumed by `SabrModelComponent` include:

- `TARGET` (required): index name (e.g. `"SOFR-1B"`).
- `VALUES` (required): one of `STANDARD`, `TOPDOWN`, `BOTTOMUP`.
- `PRODUCT` (optional): a product tag (e.g. `"CAPLET"`, `"SWAPTION"`) used to disambiguate component keys.
- `INTERPOLATION` (optional): 2D interpolation method for surfaces (default `LINEAR`).
- `SHIFT` (optional): shift applied to lognormal pricing formulas.
- `VOL_DECAY_SPEED` (optional): mean-reversion/decay parameter used by top-down/bottom-up routines.

7.4 Typical Workflow (usage example)

The tests illustrate the end-to-end workflow:

1. Build a `DataCollection` for curve calibration quotes (typically using `build_yc_data_collection`).
2. Instantiate `YieldCurve` with a list of curve build methods and calibrate on construction.
3. Build SABR parameter surfaces (`Data2D` objects) and combine them into a `DataCollection`.
4. Instantiate `SabrModel` (or `SabrModel.from_data` when starting from a unified market table).

```
from fixedincomelib.data.yc_market_loader import build_yc_data_collection
from fixedincomelib.data import Data2D, DataCollection
from fixedincomelib.yield_curve import YieldCurve
from fixedincomelib.sabr import SabrModel

# 1) Yield curve data + build methods
dc_curve = build_yc_data_collection(value_date, yc_market_df)
build_methods_yc = [{
    "TARGET": "SOFR-1B",
    "INSTRUMENTS": ["SOFR-FUTURE-3M", "USD-SOFR-OIS"],
    "INTERPOLATION METHOD": "PIECEWISE_CONSTANT",
    "LOCAL_TOL": 1e-10
}]
yc = YieldCurve(value_date, dc_curve, build_methods_yc)

# 2) SABR surfaces (example: normalvol surface)
nv = Data2D.createDataObject("normalvol", "SOFR-1B", nv_df)
beta = Data2D.createDataObject("beta", "SOFR-1B", beta_df)
nu = Data2D.createDataObject("nu", "SOFR-1B", nu_df)
rho = Data2D.createDataObject("rho", "SOFR-1B", rho_df)
```

```
dc_sabr = DataCollection([nv, beta, nu, rho])

build_methods_sabr = [{"TARGET":"SOFR-1B", "VALUES":"STANDARD",
    "INTERPOLATION":"LINEAR"}]
sabr = SabrModel(value_date, dc_sabr, build_methods_sabr)
```

Listing 7.1: Creating and calibrating models (from the testing notebooks)

Chapter 8

Product Layer

The Product layer encapsulates the definition of every financial instrument in FixedIncomeLib, from the simplest single-payment cashflow to complex multi-leg derivatives. Its primary responsibilities are:

- **Contract Modeling:** Represent each instrument's cashflow schedule, payment dates, accrual conventions, and payoff features.
- **Uniform Interface:** Provide a consistent API (`firstDate`, `lastDate`, `notional`, `longOrShort`, `currency`, `accept(visitor)`) so that valuation engines and analytics modules can treat all products generically.
- **Separation of Concerns:** Keep product definitions free of market data handling or numerical algorithms; those concerns live in other layers.
- **Extensibility:** Allow new instruments to be added with minimal changes-simply subclass the `Product` base class and implement the necessary constructors and properties.

8.1 Design Principles

Single Responsibility Each product class only carries the logic to build its own schedule of cashflows and expose its parameters; it does not perform pricing or calibration.

Composition over Inheritance Complex instruments (swaps, cap/floors, swaptions) are built by assembling simpler cashflow objects into an `InterestRateStream` or `ProductPortfolio`.

Visitor Pattern Presentation and inspection logic (e.g. rendering into tables) are delegated to concrete `ProductVisitor` implementations, keeping domain classes free of I/O concerns.

Consistent Metadata Every product carries:

- `firstDate`, `lastDate`: the schedule bounds.
- `notional` and `longOrShort`: economic direction.
- `currency`: via the `Currency` wrapper.

8.2 Core Base Class: `Product`

Defined in `product/product.py`, the abstract `Product` class establishes:

- A constructor accepting `firstDate`, `lastDate`, `notional`, `longOrShort`, and `currency`.
- Abstract method `accept(ProductVisitor)` to enable the visitor pattern.
- Read-only properties for the five core attributes.

8.3 Atomic Cash-Flow Products

8.3.1 `ProductBulletCashflow`

A zero-coupon payment at a single date. It defines only a `terminationDate` and notional transfer.

8.3.2 `ProductIborCashflow`

Captures an IBOR-based floating payment:

- `accrualStart` and `accrualEnd` dates.
- IBOR index lookup via `IndexRegistry`.
- `spread` and day-count accrual factor (using the `accrued()` utility).

8.3.3 `ProductOvernightIndexCashflow`

Similar to IBOR cashflow but supports:

- Either a fixed end date or a tenor (wrapped by `TermOrTerminationDate`).
- Compounding conventions.
- Holiday calendar adjustment for fixing and payment dates.

8.3.4 Futures: `ProductFuture` & `ProductRfrFuture`

Represent contract on forward rates:

- Strike, effective date, maturity date.
- Accrual factor computed via either contractual size or day-count.
- Underlying IBOR or OIS index obtained from the `IndexRegistry`.

8.4 Composite Instruments

8.4.1 `InterestRateStream`

A sequence of cashflows of identical type (fixed, IBOR-floating, or overnight). Built by:

1. Generating a schedule with `makeSchedule()`.
2. Instantiating the appropriate atomic cashflow for each period.
3. Collecting them into a `ProductPortfolio` base.

8.4.2 Swaps: `ProductIborSwap` & `ProductOvernightSwap`

Constructed as two `InterestRateStreams`:

- A floating leg (IBOR or OIS).
- A fixed leg (bullet repayments).

Both legs share effective and maturity dates but pay at different schedules.

8.4.3 Caps/Floors: `ProductIborCapFloorlet` & `ProductOvernightCapFloorlet`

Atomic caplet or floorlet on a single period. Composite via:

- `CapFloorStream`: sequence of caplets/floorlets.
- `ProductIborCapFloor` and `ProductOvernightCapFloor`: wrap the stream into a single `Product`.

8.4.4 Swaptions: `ProductIborSwaption` & `ProductOvernightSwaption`

Options on swaps:

- Embed an underlying swap object.
- Store an `expiryDate`.
- Delegate all cashflow inspection to the underlying swap when visited.

8.5 Product Portfolios

`ProductPortfolio` (in `product/portfolio.py`) holds any list of `(Product, weight)` pairs, enabling:

- Aggregation of cashflows across instruments.
- Priced or risk-modeled as a single composite product.

8.6 Visitor Pattern for Inspection

Implemented in `product/product_display_visitor.py`, the `ProductVisitor` hierarchy:

- Defines `visit(x: ProductType)` for each concrete product.
- Returns a `pandas.DataFrame` summarizing key attributes.
- `PortfolioVisitor` enumerates elements and weights.

8.7 Usage Example

```
from fixedincomelib.product.linear_products import ProductIborSwap
from fixedincomelib.product.product_display_visitor import IborSwapVisitor

# Instantiate a 2Y USD-LIBOR-3M pay-fixed swap
swap = ProductIborSwap(
    effectiveDate="2025-08-01",
    maturityDate="2027-08-01",
    frequency="6M",
    iborIndex="USD-LIBOR-3M",
    spread=0.0,
    fixedRate=0.0125,
    notional=50e6,
    position="SHORT"
)

# Inspect attributes
visitor = IborSwapVisitor()
df = swap.accept(visitor)
print(df)
```

Listing 8.1: Using a Visitor to Inspect a Swap

8.8 Complexity and Performance

- Atomic constructors: $O(1)$.
- Schedule generation in `InterestRateStream`: $O(N)$ for N periods.
- Visitor traversal: $O(M)$ where M is the number of attributes or cashflows visited.

8.9 Extension Guide

To add a new product:

1. Subclass `Product`, implement the constructor and `accept()`.
2. Register any new cashflow type in `utilities.makeSchedule` if needed.
3. Add a corresponding `visit()` method to a concrete `ProductVisitor`.

With this detailed structure, the Product layer remains both rich in functionality and easy to extend, while cleanly separating instrument modeling from data handling and valuation.

Chapter 9

Valuation Layer

The Valuation layer connects `Model` objects with `Product` objects by providing product-specific pricing engines. Each valuation engine:

1. computes PV (and any intermediate outputs needed for reporting), and
2. computes first-order risk by pushing derivatives back into the model gradient vector.

9.1 Core Classes

9.1.1 `ValuationEngine` (`valuation/valuation_engine.py`)

`ValuationEngine` is an abstract base class holding:

- `model`: the pricing model (e.g. `YieldCurve` or `SabrModel`)
- `product`: the instrument instance
- `valuationParameters`: a dict of additional configuration (e.g. funding index)
- `value_`: engine output tuple `(status, pv)` after `calculateValue()`
- `firstOrderRisk_`: gradient array after `calculateFirstOrderRisk()`

Concrete engines implement:

- `calculateValue()`
- `calculateFirstOrderRisk()`

9.1.2 `ValuationEngineRegistry` (`valuation/valuation_engine_registry.py`)

`ValuationEngineRegistry` provides a single entry point:

`new_valuation_engine(model, valuationParameters, product)`.

In the current codebase, dispatch is implemented as an explicit `if/elif` table over `model.modelType` and `product.prodType`, returning the corresponding engine class (for example, `ValuationEngineProductOvernightSwap` for an OIS under a `YieldCurve` model). An internal `_registry` dict and `insert()` method exist, but `new_valuation_engine` does not currently consult it; extensions therefore typically follow the existing dispatch-table pattern.

9.1.3 `IndexManager` (`valuation/index_fixing_registry.py`)

Overnight and index-based cashflows often require historical fixings to compute realized accrual. The `IndexManager` singleton:

- Loads `fixedincomelib/fixings/fixings.csv` on first use.
- Provides `getFixings(indexName)` returning an ordered dictionary of `Date -> fixing`.
- Provides `getLatestFixing(indexName)` for convenience.

Valuation engines in `yield_curve/valuation_engine_yc.py` call `IndexManager` whenever a product accrual requires realized overnight rates.

9.2 Yield Curve and SABR Engines

- **Yield curve engines** live in `yield_curve/valuation_engine_yc.py` and include engines for linear cashflows, swaps, and RFR futures.
- **SABR engines** live in `sabr/valuation_engine_sabr.py` and include engines for caplets/floorlets and swaptions, using the `SabrModel` to obtain surface parameters and then computing prices and greeks via `fixedincomelib.analytics.SABRCalculator`.

9.3 Usage Example

```
from fixedincomelib.valuation import ValuationEngineRegistry
from fixedincomelib.yield_curve import YieldCurve
from fixedincomelib.product import ProductOvernightSwap

yc = YieldCurve(value_date, dc, build_methods)

ois = ProductOvernightSwap(...)
params = {"FUNDING INDEX": "SOFR-1B"}

ve = ValuationEngineRegistry().new_valuation_engine(yc, params, ois)

ve.calculateValue()
status, pv = ve.value_

ve.calculateFirstOrderRisk()
g = ve.firstOrderRisk_  # dPV/d(theta) in model state-variable space
```

Listing 9.1: Creating a valuation engine and computing PV and first-order risk

For reporting convenience, `utilities/risk_reporting.py` provides `create_value_report` and `create_risk_report`, which wrap the registry, engine calls, and Jacobian-based mapping in a single function.

Chapter 10

Yield Curve Layer

The Yield Curve layer provides a calibrated term structure that supports discounting and forward-rate projections. In FixedIncomeLib, the concrete model is `YieldCurve` (in `yield_curve/yield_curve_model.py`), whose components calibrate piecewise-constant instantaneous forward-rate (IFR) buckets to a set of market calibration instruments.

10.1 Key Responsibilities

1. Convert curve calibration quotes in `DataCollection` into concrete calibration instruments.
2. Bootstrap IFR state variables sequentially so that each calibration instrument matches its market quote.
3. Provide query methods (`discountFactor`, `forward`) used by valuation engines.
4. Provide Jacobians and quote sensitivities for first-order risk mapping.

10.2 Core Types

10.2.1 `CalibrationBasket` (`yield_curve/calibration_basket.py`)

Curve calibration instruments are stored as a `CalibrationBasket` of `CalibItem` objects:

- `product`: a concrete `Product` (e.g. RFR future, OIS swap)
- `quote`: market quote (unit depends on instrument convention)
- `data_type`, `data_convention`, `axis`: metadata from the input market table

10.2.2 `PillarNode` (`yield_curve/pillar_node.py`)

Bootstrapping proceeds along a chain of `PillarNode` objects, each holding:

- `pillar_index`: index into the curve pillar array
- `pillar_date` and `pillar_time`: anchor date and time-to-anchor
- `product` and `quote`: the calibration instrument tied to this pillar
- `state_value`: the IFR parameter value for this bucket

10.2.3 Builders: from quotes to products (`builders/`)

The curve does not calibrate to "zero rates" in this repository; it calibrates to traded instruments. The conversion from quotes to products is handled by the `builders` package:

- `builders/basket_builders.py`: `build_yc_calibration_basket_from_dc(...)` creates a `CalibrationBasket` using the `INSTRUMENTS` list from the build method.
- `builders/product_builder_registry.py` and `builders/instrument_builders.py`: select a builder based on `DataConvention.kind` and instantiate the correct `Product` for each axis/quote.
- `builders/pillar_builders.py`: generates anchor pillars by querying each built product for its terminal date.

10.3 Curve Representation

Let $\{T_0 = \text{valueDate} < T_1 < \cdots < T_n\}$ be the pillar dates for a given curve component. The model stores a vector of IFR bucket values

$$\theta = (\theta_1, \ldots, \theta_n),$$

where θ_k applies on $(T_{k-1}, T_k]$.
The discount factor from $t = T_0$ to a maturity T is computed by integrating the IFR buckets. In the implementation, `YieldCurveModelComponent.discountFactor(maturityDate)`:

1. converts T to a time u (year fraction from value date),
2. identifies which buckets are covered up to u,
3. sums $\int f\,dt$ over covered buckets, and
4. returns $D(T) = \exp(-\int_0^u f(s)\,ds)$.

The forward rate between two dates is then derived from discount factors and an accrual basis:

$$F(t_1, t_2) = \frac{D(t_1)}{D(t_2)} - 1 \Big/ \text{yearFraction}(t_1, t_2).$$

The library implements this in `YieldCurve.forward(startDate, endDate, accrualBasis, biz_conv, hol_conv)`, applying business-day adjustments to the end date before computing the accrual fraction.

10.4 Calibration Algorithm

Calibration occurs inside `YieldCurveModelComponent.calibrate()`:

10.4.1 Step 1: Build calibration basket

The method calls:

```
build_yc_calibration_basket_from_dc(value_date, data_collection, build_method)
```

to select instrument quote series from the `DataCollection`, resolve each `DATA CONVENTION` via `DataConventionRegistry`, and build a `CalibrationBasket` of `CalibItem(product, quote, ...)`.

10.4.2 Step 2: Build anchor pillars

Anchor dates are constructed by:

`build_anchor_pillars(calibrationBasket, max_pillar_date)`.

Each calibration instrument provides a terminal date; these are converted into a strictly increasing list of pillar dates and corresponding times.

10.4.3 Step 3: Sequential solve of IFR buckets

For each pillar node k (in increasing maturity order), define a residual function:

$$r_k(\theta_k) = PV_k(\theta_1, \ldots, \theta_{k-1}, \theta_k) - PV_k^{mkt},$$

where $PV_k(\cdot)$ is the model PV of the k-th calibration instrument with the curve defined by the current IFR vector and PV_k^{mkt} is the target PV implied by the market quote.
The implementation constructs r_k by:

1. temporarily setting the curve component state variables up to k,
2. creating a valuation engine for the calibration product,
3. valuing the instrument, and
4. returning a quote-matching residual.

The root solve uses `utilities.simple_solver` (secant method), with local tolerance `LOCAL_TOL` from the build method. After solving, θ_k is frozen and the algorithm proceeds to $k + 1$.

10.5 Risk: Jacobian and Quote Sensitivities

First-order risk reporting relies on two model-level objects:

10.5.1 `jacobian()`

`YieldCurve.jacobian()` builds the matrix

$$J_{i,j} = \frac{\partial PV_i}{\partial \theta_j},$$

where PV_i is the PV of the i-th calibration instrument and θ_j is the j-th IFR bucket. Each row is computed by calling `calculateFirstOrderRisk()` on the corresponding calibration instrument under the calibrated curve.

10.5.2 `calibration_quote_sensitivity()`

Calibration instruments are quoted in market units (rates, swap fixed rates, *etc.*).
`YieldCurve.calibration_quote_sensitivity()` returns a vector

$$s_i = \frac{\partial PV_i}{\partial q_i},$$

so that mapped risk can be expressed in quote units when desired.
`utilities/risk_reporting.py` uses this scaling when producing quote-space risk reports.

10.6 Usage Example

```
from fixedincomelib.data.yc_market_loader import build_yc_data_collection
from fixedincomelib.yield_curve import YieldCurve

# market_df: columns [DATA TYPE, DATA CONVENTION, AXIS, VALUE]
dc = build_yc_data_collection(value_date, market_df)

build_methods = [{
    "TARGET": "SOFR-1B",
    "INSTRUMENTS": ["SOFR-FUTURE-3M", "USD-SOFR-OIS"],
    "LOCAL_TOL": 1e-10
}]

yc = YieldCurve(value_date, dc, build_methods)

df_2y = yc.discountFactor("2028-02-08")
fwd_3m = yc.forward("2026-02-08", "2026-05-08", accrualBasis="ACT/360")
```

Listing 10.1: Yield curve calibration and basic queries

Chapter 11

SABR Layer

The SABR layer provides a parametric volatility description used for non-linear rate products (caplets/floorlets and swaptions). In this repository, `SabrModel` does *not* calibrate SABR parameters from option prices; instead it **ingests pre-built parameter surfaces** and provides fast interpolation plus a standardized gradient layout for valuation engines.

11.1 Core Classes

11.1.1 `SabrModel` (`sabr/sabr_model.py`)

`SabrModel` subclasses `Model` and builds one or more `SabrModelComponent` objects from `buildMethodCollection`. Component keys follow:

$$\texttt{componentKey} = \texttt{index} \; + \; \texttt{"-"} \; + \; \texttt{VALUES} \; + \; [\texttt{"-"} + \texttt{PRODUCT}],$$

so the same index can support multiple components (e.g. `SOFR-1B-STANDARD`, `SOFR-1B-TOPDOWN-CAPLET`).

11.1.2 `SabrModelComponent`

A `SabrModelComponent` reads four surfaces from the `DataCollection`:

- `normalvol` (ATM normal volatility surface),
- `beta`,
- `nu`,
- `rho`,

all keyed by `(data_type, data_convention) = (param.lower(), TARGET)`.
For each surface, the component builds and stores:

- the axis1 grid, axis2 grid, and value grid,
- a 2D interpolator (`Interpolator2D`), and
- interpolation weights returned by `Interpolator2D.interpolationWeights(...)` used later for gradient mapping.

The component also stores two scalars from the build method:

- `shift` (`SHIFT`, default 0),
- `vol_decay_speed` (`VOL_DECAY_SPEED`, default 0).

11.2 Interpolation API

11.2.1 `get_sabr_parameters`

The method

```
get_sabr_parameters(index, VALUES, optionMat, optionTenor, product=None)
```

returns a 6-tuple:

(`normalvol`, β, ν, ρ, `shift`, `vol_decay_speed`),

where the first four entries are bilinear interpolations on the stored parameter grids.

11.2.2 `get_weights`

`get_weights(...)` returns the interpolation weights used to map a scalar sensitivity at `(optionMat, optionTenor)` back to the four corner grid nodes for each surface.

11.3 State Variables and Gradient Layout

`SabrModelComponent.calibrate()` flattens each parameter grid into the component's `stateVars_` vector (with a fixed parameter ordering). This ordering is mirrored when the component writes into the model gradient array, so that valuation engines can compute:

$$\frac{\partial PV}{\partial(\text{surface node})}.$$

The stored `weights_` objects provide the sparse mapping from a single `(mat, tenor)` evaluation point back to surrounding grid nodes.

11.4 Usage Example

```
from fixedincomelib.data import Data2D, DataCollection
from fixedincomelib.sabr import SabrModel

nv   = Data2D.createDataObject("normalvol", "SOFR-1B", nv_df)
beta = Data2D.createDataObject("beta",      "SOFR-1B", beta_df)
nu   = Data2D.createDataObject("nu",        "SOFR-1B", nu_df)
rho  = Data2D.createDataObject("rho",       "SOFR-1B", rho_df)

dc = DataCollection([nv, beta, nu, rho])

build_methods = [{
    "TARGET": "SOFR-1B",
    "VALUES": "STANDARD",
    "INTERPOLATION": "LINEAR",
    "SHIFT": 0.0,
    "VOL_DECAY_SPEED": 0.0
}]
sabr = SabrModel(value_date, dc, build_methods)

params = sabr.get_sabr_parameters("SOFR-1B", "STANDARD", optionMat=1.0, optionTenor=5.0)
```

Listing 11.1: Building a SABR model component from parameter surfaces

Chapter 12

Analytics Layer

The Analytics layer implements reusable quantitative routines that sit *on top of* the model and interpolation layers. In FixedIncomeLib, these routines primarily support:

- SABR pricing and greeks (via `SABRCalculator`),
- RFR-specific aggregation schemes ("top-down" and "bottom-up") that transform input SABR parameter surfaces into effective parameters over a coupon period, and
- correlation-surface interpolation for the bottom-up scheme.

12.1 Unified SABR Calculator (`analytics/sabr_calculator.py`)

`SABRCalculator` wraps `pysabr`'s `Hagan2002LognormalSABR` implementation and standardizes:

- conversion between "normalvol" inputs and a lognormal-SABR α when needed,
- option pricing under Black(-76) with optional shift,
- partial derivatives of the implied volatility with respect to SABR parameters, and
- chaining those derivatives into PV greeks.

The entry points are:

- `option_price(...)`: returns an option PV given (F, K, T) and SABR inputs.
- `option_price_greeks(...)`: returns PV plus a dict of greeks with respect to the SABR inputs.

The calculator supports multiple methods:

- `method=None`: standard lognormal Hagan SABR.
- `method="TOPDOWN"`: apply a time-decay mapping (Section 12.3).
- `method="BOTTOMUP"`: apply a coupon-aggregation mapping (Section 12.4).

12.2 Correlation Surface (`analytics/correlation_surface.py`)

The bottom-up mapping uses an interpolated correlation surface:

- stored as a `Data2D` object with `data_type="corr"` and `data_convention=<index>`,
- wrapped in `CorrSurface`, which builds an `Interpolator2D` and exposes:

`get_corr(expiry, tenor)`.

12.3 Top-down time-decay mapping

`analytics/sabr_top_down.py` defines `TimeDecayLognormalSABR`, which modifies SABR parameters to account for volatility decay between a "decay start" time and a coupon end time. In the current implementation, the pricer is constructed with:

$$t = t_{\text{end}}, \quad t_s = t_{\text{start}}, \quad t_e = t + t_s,$$

where t_{start} and t_{end} are times (year fractions) used by the calling valuation engine.
Given base parameters $(\alpha_0, \beta, \nu_0, \rho_0)$ and decay speed k, the code computes two denominators:

$$\texttt{den0} = 2\,t^3(2k+1), \qquad \texttt{den2} = t(1+2k)^2(1+4k),$$

then forms:

$$\gamma = \frac{t\,(2t^3 + t_e^3 + (4k^2 - 2k)t_s^3 + 6k\,t_s^2 t_e)}{\texttt{den0}} + \frac{3k\rho_0^2(t_e - t_s)^2\,(3t^2 - t_e^2 + 5k\,t_s^2 + 4t_s t_e)}{\texttt{den2}},$$

and uses it to define effective parameters:

$$\nu_{\text{eff}}^2 = \nu_0^2\,\gamma\,\frac{2k+1}{t^3\,t_e},$$

$$H = \nu_0^2 \frac{t^2 + 2k\,t_s^2 + t_e^2}{2\,t_e\,t\,(k+1)} - \nu_{\text{eff}}^2,$$

$$\alpha_{\text{eff}}^2 = \frac{\alpha_0^2}{2k+1}\left(\frac{t}{t_e}\right)\exp\!\left(\tfrac{1}{2}Ht_e\right),$$

$$\rho_{\text{eff}} = \rho_0 \frac{3t^2 + 2k\,t_s^2 + t_e^2}{\sqrt{\gamma}\,(6k+4)}.$$

These expressions are implemented directly in `TimeDecayLognormalSABR._computeEffectiveParams()`, with runtime checks enforcing $\gamma > 0$ and $|\rho_{\text{eff}}| < 1$.

12.4 Bottom-up coupon aggregation

`analytics/sabr_bottom_up.py` defines `BottomUpLognormalSABR`, which aggregates SABR parameters across a set of coupon sub-periods. The caller provides arrays:

$$\{\alpha_i\},\ \{\beta_i\},\ \{\nu_i\},\ \{\rho_i\},$$

together with:

- coupon weights `weights` (one per sub-period),
- segment lengths `segment_lengths`,
- time scales `time_scales`, and
- a correlation surface `CorrSurface` providing ρ_{1N} at `(expiry, tenor_total)`.

The implementation constructs:

$$\mu = \frac{1-\rho_{1N}}{N-1}, \qquad \Gamma_{ij} = \max\{0,\, 1-\mu\,|i-j|\}, \qquad \bar{\gamma} = \texttt{mean}(\Gamma),$$

where i, j index sub-periods (note the discrete index distance $|i-j|$; this matches the shipped code). It then computes weighted effective parameters in `effective_sabr_parameters_over_period(...)`:

- effective β as a weight-normalized average,
- effective α using time-scaling and the correlation adjustment $\bar{\gamma}$,
- effective ν using a correlation-adjusted second moment,
- effective ρ using a correlation-adjusted weighted average.

12.5 Risk Propagation to Surface Nodes

Valuation engines call `SabrModel.get_weights(...)` to obtain bilinear interpolation weights for each underlying surface (`normalvol`, `beta`, `nu`, `rho`). If an engine computes a scalar sensitivity $\partial PV/\partial p$ at a point $(\text{mat}, \text{tenor})$, the weights provide the sparse mapping back to the four surrounding nodes:

$$\frac{\partial PV}{\partial p_{i,j}} = w_{i,j}\ \frac{\partial PV}{\partial p(\text{mat}, \text{tenor})},$$

with $\sum w_{i,j} = 1$ over the relevant corners. This ensures the model gradient vector is aligned with the flattened `stateVars_` ordering inside `SabrModelComponent`.

Chapter 13

Risk Interpretation and Hedge Vectors

This chapter makes the "risk plumbing" explicit: what a reported risk vector means, what space it is in, and how to interpret it as hedges.

13.1 Three spaces: model parameters, calibration instruments, and quotes

FixIncomeLib works with three closely-related first-order objects.

(1) Model-parameter risk: $g = \partial PV/\partial\theta$

A valuation engine computes the gradient of product PV with respect to model state variables θ. Examples:

- Yield curve: θ are IFR bucket levels (one per pillar interval).
- SABR: θ are surface nodes (flattened grids) for `NORMALVOL`, `BETA`, `NU`, `RHO` (optionally per product-type variant).

This vector lives in "internal model coordinates." It is the most fundamental risk object because it is what pricing depends on directly.

(2) Calibration-instrument hedge vector: solve $J^\top r = g$

Let $V_i(\theta)$ be the PV of the i-th *calibration instrument* (future, swap, ...) under the calibrated model. Define the Jacobian

$$J_{i,j} = \frac{\partial V_i}{\partial \theta_j}.$$

If we hold $r = (r_1, \ldots, r_n)$ units of calibration-instrument PVs, then the portfolio PV sensitivity to θ is:

$$\frac{\partial}{\partial\theta}\left(\sum_i r_i V_i(\theta)\right) = J^\top r.$$

Therefore, solving

$$J^\top r = g$$

produces a **replicating first-order hedge** in the space of calibration instruments: the calibration-instrument PV portfolio whose θ-sensitivities match the product's θ-sensitivities. In code, `utilities/risk_reporting.py` computes r via `np.linalg.solve(J.T, g)`.

(3) Quote-space sensitivities (PV01 / bucketed risk)

Calibration instruments are quoted in market units q_i (e.g., futures price, fixed swap rate). The model provides a per-instrument scale:

$$S_i = \frac{\partial V_i}{\partial q_i},$$

implemented by `YieldCurve.calibration_quote_sensitivity()`. Quote-space hedge numbers are then:

$$r_i^{(q)} = \frac{r_i}{S_i},$$

which has units "PV per 1.0 quote unit." For PV01-style interpretation, multiply by 10^{-4} (one basis point) when the quote is a rate.

13.2 How to read risk outputs

Typical outputs you will see from `createRiskReport`:

- `g`: "bucketed model risk" (IFR bucket risk or surface-node risk).
- `r`: "calibration hedge vector" (weights in calibration instruments).
- `r\_quote`: "quote PV01s" (or quote sensitivities) after scaling by S_i.

Practical interpretation:

- For yield curves, `r\_quote` aligns with a bucketed PV01 profile across the curve pillars tied to your calibration set.
- For SABR surfaces, surface-node risk is naturally sparse around the queried (expiry, tenor) point (bilinear weights), and can be aggregated into expiry buckets or tenor buckets for reporting.

Chapter 14

How to extend the library

This chapter documents the highest-value extension hooks: adding a new curve instrument, adding a new product, and adding a new valuation engine. The guiding principle is: **extend via registries and new classes, not by editing core logic.**

14.1 Add a new curve calibration instrument

A "curve instrument" is something that can appear in the market table and be turned into a product used for curve calibration.

1. **Add or update an instrument convention** in `conventions/data_convention_registry.json`. Choose an existing `kind` if possible; otherwise add a new `kind` and extend the factory.
2. **Ensure the convention factory supports the kind**: update `conventions/data_conventions.py` so `DataConventionFactory.createDataConvention` returns the correct `DataConvention` subclass.
3. **Implement a builder**: add a function in `builders/instrument_builders.py` that converts `(valueDate, axis, quote, convention)` into a `Product` instance.
4. **Register the builder**: call `ProductBuilderRegistry().insert(ConventionSubclass, builderFn)` (the repo registers builders at import time in `instrument_builders.py`).
5. **Ensure a valuation engine exists** for that product under a `YieldCurve` model (so it can be used during calibration). If needed, implement a new engine in `yield_curve/valuation_engine_yc.py` and register it in `ValuationEngineRegistry`.

14.2 Add a new product

1. Create a new `Product` subclass in `fixedincomelib/product/`. Give it a distinct `prodType` string.
2. Implement its constructor and `accept(visitor)`.
3. Add (optional) inspection support by extending `product/product_display_visitor.py`.
4. Implement a valuation engine for the new `prodType` under the relevant model type(s) and register it (next section).

14.3 Add a new valuation engine

1. Implement a new engine subclass of `ValuationEngine` in the appropriate module (e.g., `yield_curve/valuation_engine_yc.py` or `sabr/valuation_engine_sabr.py`).
2. Implement `calculateValue()` and `calculateFirstOrderRisk()`.
3. Register the engine:

```
from fixedincomelib.valuation.valuation_engine_registry import ValuationEngineRegistry

ValuationEngineRegistry.insert(
    modelType="YIELD_CURVE",
    prodType="MyNewProductReveal",
    ve=ValuationEngineMyNewProduct
)
```

Listing 14.1: Registering a new valuation engine

Common pitfall: registration happens at import time. Make sure the module defining and registering your engine is imported before calling `new_valuation_engine`.